

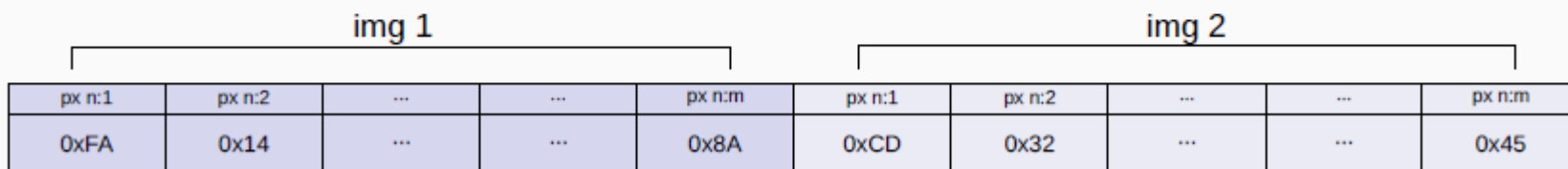
CAMADA FÍSICA DA COMPUTAÇÃO

AULA 5 – Datagrama

Prof. Rodrigo Carareto

Streaming

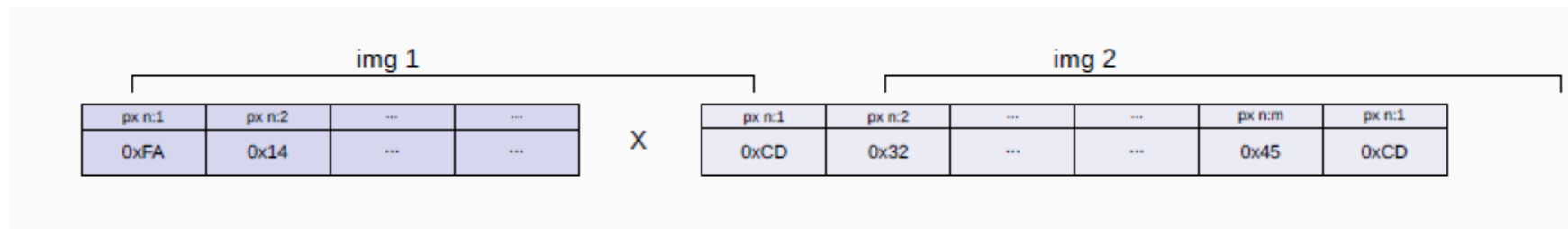
O que aconteceria no protocolo de vocês se precisássemos enviar duas imagens em sequência ?



Esse tipo de envio é denominado streaming.

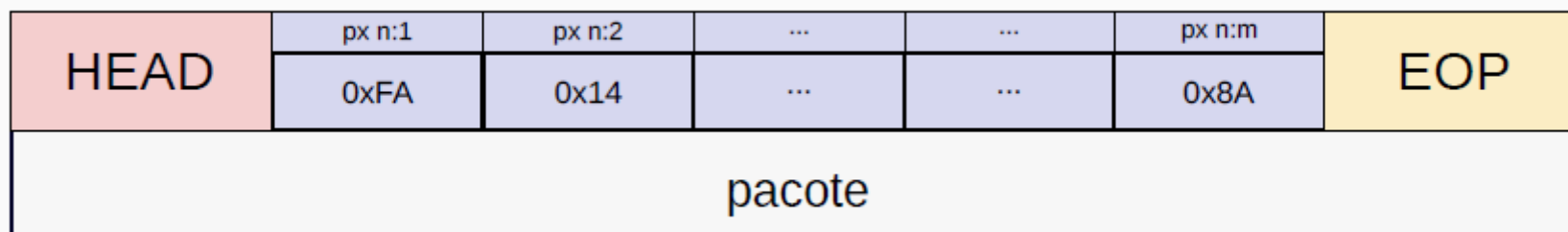
- Vantagens - Velocidade
- Desvantagens - Não há garantia de entrega.

Perda de bytes no envio

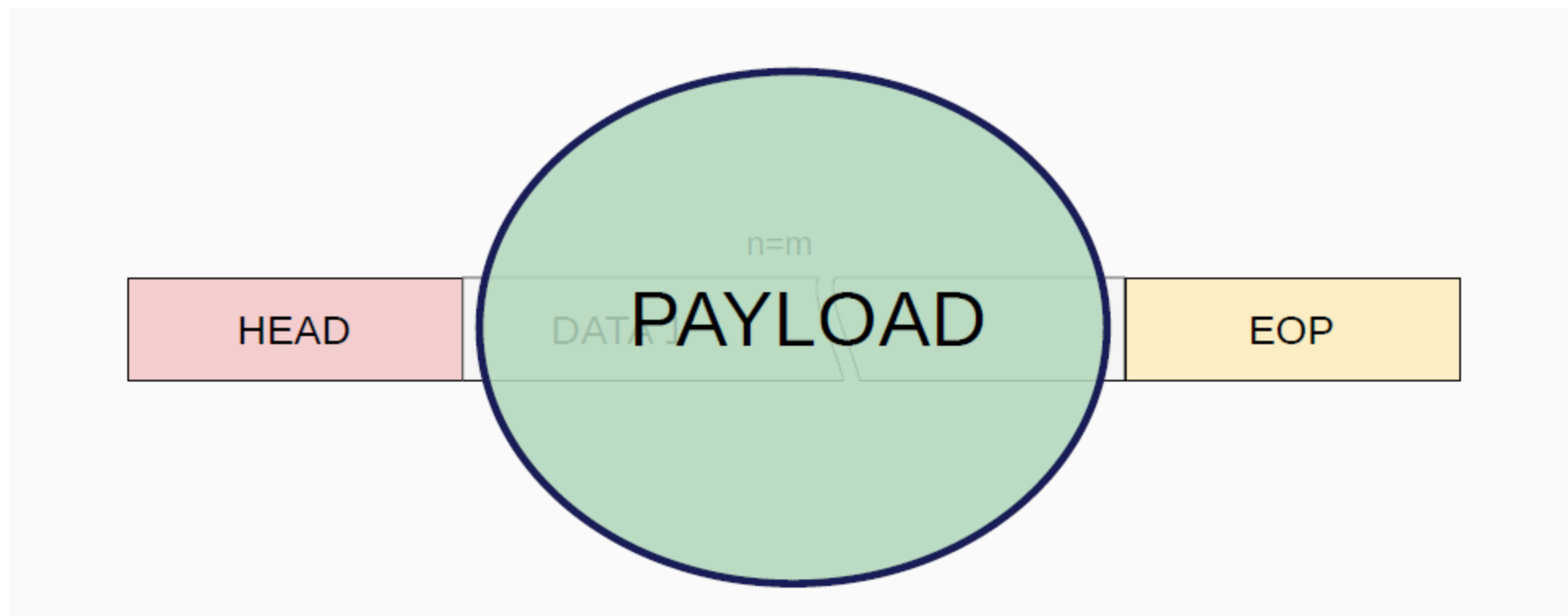


As imagens iriam se misturar!

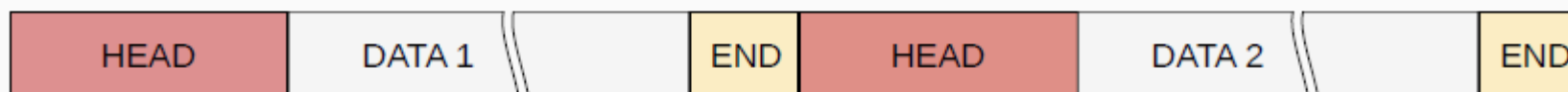
Empacotamento



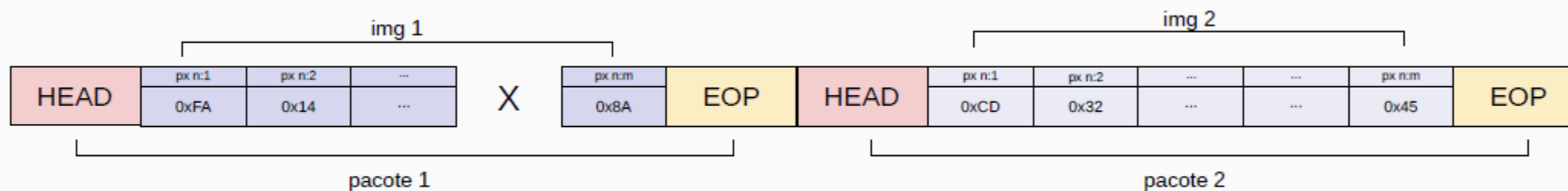
0 pacote



Pacote



Envio em pacotes



TIPICAMENTE UM HEAD CONTEM:

- Tipo de pacote (dados, comando etc.)
- Versão (IPv4, IPv6)
- Número do pacote (incremental durante a transmissão)
- Tamanho do dado que o pacote pode transmitir
- O destinatário
- A origem
- etc

DATAGRAMA DO TCO/IP

Cabeçalho de uma trama TCP

+	Bits 0 - 3	4 - 9	10 - 15	16 - 31
0	Porta na origem			Porta no destino
32	Número de sequência			
64	Número de confirmação (ACK)			
96	Offset	Reservados	Flags	Janela Window
128	Checksum			Ponteiro de urgência
160	Opções (opcional)			
	Padding (até 32)			
224	Dados			

EOP – End of package

- Sequência de bytes conhecida para ser possível a identificação
- Possível verificação.

Payload e OverHead

$$\text{OverHead} = \frac{\text{tamanho total}}{\text{pay load}} = \frac{\text{head} + \text{pay load} + \text{eop}}{\text{pay load}}$$

Taxa de envio e download

ThroughPut: Diferentes taxas para transmissão e recepção

Definição de datagrama

“Uma entidade independente e autocontida de dados que transporta informações suficientes para serem roteadas da origem ao computador de destino sem depender de trocas anteriores entre este computador de origem e de destino e a rede de transporte”.

— RFC 1594 [↗](#)

Exemplos

UDP

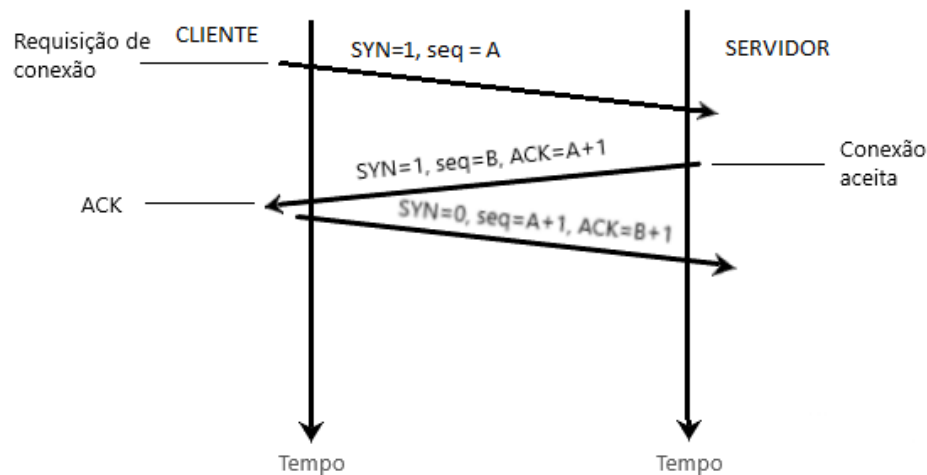
O **User Datagram Protocol** (UDP) é um [protocolo](#) simples da [camada de transporte](#). Ele é descrito na [RFC 768](#) e permite que a aplicação envie um [datagrama](#) encapsulado num pacote [IPv4](#) ou [IPv6](#) a um destino, porém sem qualquer tipo de garantia que o pacote chegue corretamente (ou de qualquer modo).^[1]

TCP

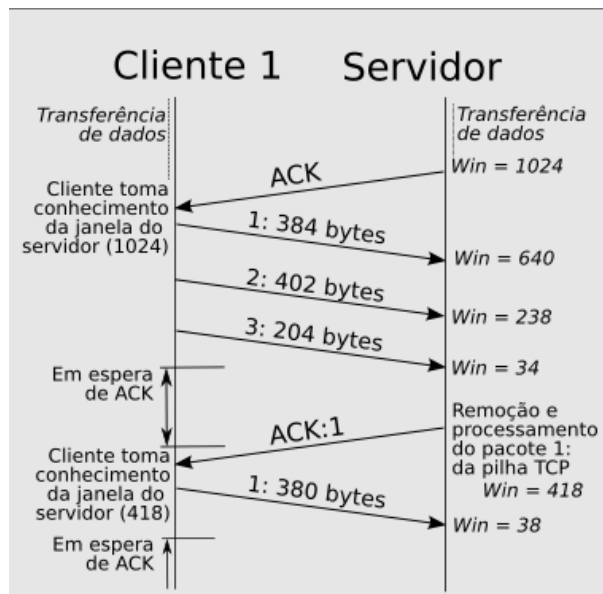
Em comparação ao [TCP](#), é possível entendê-lo como um envio de cartas pelo correio, onde o usuário escreve a carta, envelope como o endereço de origem e destino, envia, mas não consegue ter a confirmação imediata se aquilo chegou ou não ao destino, ele só tem certeza do envio. Já o TCP pode ser tido como um telefone, onde é possível saber de imediato se o destinatário está ou não recebendo as informações. ^[2]

Exemplos

TCP



- **Orientado à conexão**
- **Hand-shake**
- **Ponto a ponto**
- **Confiabilidade**
- **Full duplex**
- **Controle de fluxo**
- **Controle de congestionamento**



+	Bits 0 - 3	4 - 9	10 - 15	16 - 31
0	Porta na origem			Porta no destino
32	Número de sequência			
64	Número de confirmação (ACK)			
96	Offset	Reservados	Flags	Janela Window
128	Checksum			Ponteiro de urgência
160	Opções (opcional)			
Padding (até 32)				
224	Dados			

Detalhe do campo *Flags*

+	10	11	12	13	14	15
96	<i>UrgPtr</i>	<i>ACK</i>	<i>Push</i>	<i>RST</i>	<i>SYN</i>	<i>FIN</i>

"Hi, I'd like to hear a TCP joke."
"Hello, would you like to hear a TCP joke?"
"Yes, I'd like to hear a TCP joke."
"OK, I'll tell you a TCP joke."
"Ok, I will hear a TCP joke."
"Are you ready to hear a TCP joke?"
"Yes, I am ready to hear a TCP joke."
"Ok, I am about to send the TCP joke. It will last 10 seconds, it has two characters, it does not have a setting, it ends with a punchline."
"Ok, I am ready to get your TCP joke that will last 10 seconds, has two characters, does not have an explicit setting, and ends with a punchline."
"I'm sorry, your connection has timed out."
...Hello, would you like to hear a TCP joke?"

TCP



UDP



PROJETO 3:

handshake – fragmentação – ack/nack

FUNCIONALIDADES:

- **Nesse projeto, seu datagrama não pode ultrapassar 65 bytes.** Consequentemente, como já dito antes, seu payload será menor que isso, exigindo uma fragmentação da imagem a ser feita pelo cliente.
- Quando o client enviar um pacote, deve informar obrigatoriamente (em algum espaço do head reservado a isso), o número do pacote e o número total de pacotes que serão transmitidos.
- Ao receber um pacote, o server deve fazer duas verificações: verificar que o número do pacote é 1 a mais que o anterior, ou seja, a ordem está correta. Deve também verificar se o EOP está no local correto (vieram todos os bytes).
- Se tudo estiver ok, o server deve enviar uma mensagem ao cliente para que este envie o próximo pacote.
- Após o envio de um pacote o cliente não envia o próximo enquanto não receber a confirmação do server de recebimento do pacote enviado.
- Se algo estiver errado, o server deve enviar uma mensagem para o cliente solicitando o reenvio do pacote, seja por não ter o payload esperado, ou por não ser o pacote correto.
- Ao receber o último pacote o Server deve ser capaz de reagrupá-los e salvar o arquivo em seu estado original. Após o reagrupamento, o server faz uma última resposta ao cliente, informando que a transmissão foi feita com sucesso.
- Quando houver algum erro de transmissão, o client não precisa corrigir o erro e continuar a transferência de dados. A comunicação deve se encerrar.